
Agile Project Documentation

AI Career Companion Agent for Internship Matching
and Interview Preparation

Project Title:	AI Career Companion Agent for Internship Matching and Interview Preparation
Prepared By:	Rishi Yadav
Project Duration:	22 July – 22 September 2026
Mentor:	Vijay Parmar
Tech Stack:	FastAPI · Next.js · Groq · FAISS · PostgreSQL
GitHub:	github.com/RishiY7/AI-Career-Companion-Agent

Contents

1	Project Overview	1
1.1	Project Name	1
1.2	What is the Project?	1
1.3	Problem Statement	1
1.4	Target Users	2
1.5	Project Scope	2
2	Project Objective	2
2.1	Success Metrics	2
3	Technology Stack	3
4	Project Requirements	4
4.1	Functional Requirements	4
4.2	Non-Functional Requirements	5
5	System Architecture / Workflow	5
5.1	High-Level Architecture	5
5.2	Resume Upload & Matching Flow	6
5.3	RAG / AI Processing Workflow	7
5.4	Interview Preparation Agent – Intent Flow	8
5.5	Application Tracker Flow	8
6	Product Backlog	8
7	Sprint Planning	9
8	Sprint Tasks & Status	11
8.1	Task Kanban Flow	11
8.2	Task Tracker	11
9	Implementation / Key Features	13
9.1	Feature 1: User Authentication	13
9.2	Feature 2: Resume Upload & LLM Parsing	14
9.3	Feature 3: Internship Matching (FAISS + LLM)	14
9.4	Feature 4: Cover Letter & Skill-Gap Generation	15
9.5	Feature 5: Product Chatbot (RAG)	15
9.6	Feature 6: Interview Preparation Agent	16
9.7	Feature 7: Application Tracker	16
10	Testing	17
10.1	Test Cases	17
10.2	Testing Types Performed	18
11	Challenges & Solutions	18
12	Final Project Status	20
12.1	Completed Features	20
12.2	Final Outcome	20

1 Project Overview

1.1 Project Name

AI Career Companion Agent for Internship Matching and Interview Preparation

1.2 What is the Project?

I built the **AI Career Companion Agent** over nine weeks as my internship project — a full-stack web application that tries to solve a genuinely frustrating problem: the gap between a student's resume and landing an internship they're actually suited for.

At its core, the platform takes a candidate's PDF resume, uses a **Groq LLM** to parse it into structured data, and then runs a **FAISS** semantic search to find the internships that genuinely match their skills — not just keyword matches, but actual semantic alignment. The **FastAPI** backend handles all the AI-heavy lifting while a **Next.js 14** frontend gives users a clean, responsive interface to see their results in real time.

But the matching is just the starting point. Once a candidate spots an internship they like, they can generate a personalised cover letter (powered by **Google Gemini**), get a brutally honest skill-gap analysis, chat with a **RAG-based product chatbot** about how the platform works, and — the feature I'm most proud of — run a full **Interview Preparation session** where the AI asks them mock technical and HR questions tailored to their own resume and target role.

1.3 Problem Statement

When I started looking for internships myself, I ran into the same three walls that most students hit:

1. **Discovery Gap:** Job portals are full of noise. Keyword-based search returns roles you're nowhere near qualified for or misses ones you'd be perfect for — because it's matching words, not meaning.
2. **Preparation Gap:** Real interview coaching is either expensive or too generic. Watching YouTube prep videos doesn't help when you don't know which roles suit you or what questions to expect for your specific background.
3. **Application Gap:** Writing a tailored cover letter for every internship is exhausting, and most candidates have no idea which skills they're actually missing for a target role until they get rejected.

This project is my answer to all three — one platform that understands your resume deeply and guides you through the entire internship journey, from finding the right role to walking into the interview prepared.

1.4 Target Users

#	User Type	Description
1	Students	Undergraduate and postgraduate students seeking summer or semester internships in technology and engineering domains.
2	Early-career professionals	Candidates with fewer than two years of work experience looking to transition into new roles or companies via internship pathways.
3	Admins	Platform administrators responsible for maintaining internship listings, monitoring system health, and updating the product knowledge base.

1.5 Project Scope

In Scope: Resume upload and LLM parsing; FAISS-based internship matching; cover letter and skill-gap generation; product chatbot (RAG); interview preparation agent; application tracker; full Next.js dashboard.

Out of Scope: Real-time job board scraping; OAuth / third-party login; mobile app; email notifications; employer-side portal.

2 Project Objective

From the start, I set myself clear, measurable goals rather than vague ambitions. The table below captures exactly what I committed to building — and as of Sprint 9, every one of them is done.

#	Objective
1	Automate resume parsing: extract name, skills, education, experience, and projects into a structured JSON profile using a Groq LLM.
2	Match candidates to the top- <i>k</i> relevant internships from a curated dataset using FAISS semantic search combined with an LLM-generated ranking rationale.
3	Enable personalised cover letter generation (Gemini) and skill-gap analysis (Groq) for any selected internship.
4	Provide a multi-intent Interview Preparation Agent supporting role recommendation, mock technical and HR Q&A with answer guidance, and document-based Q&A from uploaded PDFs or DOCX files.
5	Deliver a RAG-based product chatbot that answers platform-related queries using a curated product knowledge base indexed in FAISS.
6	Allow candidates to track their internship applications through a full status pipeline: <i>Applied</i> → <i>Interviewing</i> → <i>Offered/Rejected</i> .

2.1 Success Metrics

- Resume parsed and matched within **30 seconds** of upload.
- FAISS retrieval returns semantically relevant results for $\geq 90\%$ of test resumes.

- Cover letter and skill-gap generated in <10 seconds per request.
- Interview agent correctly classifies intent for $\geq 95\%$ of test queries.
- All 11 functional requirements verified and passing by project end.

3 Technology Stack

Category	Technology	Justification
Programming Language	Python 3.11+	Dominant language for AI/ML; rich ecosystem of LLM and vector-search libraries.
Backend Framework	FastAPI	Async-first, auto-generated OpenAPI docs, native BackgroundTasks for decoupled matching.
Frontend Framework	Next.js 14 (App Router, TypeScript)	Server components, file-based routing, and Tailwind CSS integration for rapid UI development.
Primary LLM	Groq (compound-mini)	Ultra-low latency inference (<500 ms) on open-weight models; ideal for real-time chat and parsing.
Secondary LLM	Gemini 2.5 Flash	Used for cover letter generation (higher quality output) and as automatic fallback when Groq is unavailable.
AI Orchestration	LangChain / LangChain-Core	Provides LCEL chains, retriever abstractions, memory, and <code>.with_fallbacks()</code> for multi-LLM reliability.
Embeddings	HuggingFace Sentence Transformers	Free, offline-capable dense embeddings; consistent quality for semantic similarity tasks.
Vector Database	FAISS (CPU)	In-process, zero-latency similarity search; no external service required; persists as a local index file.
Relational Database	PostgreSQL 15	Robust, open-source RDBMS; strong SQLAlchemy support; reliable ACID guarantees.
ORM	SQLAlchemy 2.x	Declarative models, session management, and alembic-compatible migrations.
Styling	Tailwind CSS	Utility-first CSS; eliminates context-switching between component and stylesheet files.
Authentication	passlib[bcrypt] v1.7.4	Industry-standard password hashing; no JWT overhead required for this scope.
Document Parsing	PyMuPDF + python-docx	PyMuPDF (fitz) gives clean PDF text extraction; python-docx handles DOCX interview documents.
Version Control	Git / GitHub	Standard collaborative development; used for sprint branching and code review.

LLM Fallback Strategy

`get_llm()` in `llm_client.py` returns a Groq LLM wrapped with `.with_fallbacks([gemini])`. If Groq raises any exception (rate-limit, timeout, model error), LangChain transparently retries the same prompt on Gemini — zero manual error handling required in feature code.

4 Project Requirements

4.1 Functional Requirements

ID	Requirement
FR-01	The system shall allow a user to register with a unique email address and a password that is stored as a bcrypt hash.
FR-02	The system shall authenticate users via email and password and return a persistent <code>user_id</code> for session tracking.
FR-03	The system shall accept PDF resume uploads and extract raw text using PyMuPDF.
FR-04	The system shall parse the extracted resume text into a structured JSON profile (name, skills, education, experience, projects) using a Groq LLM.
FR-05	The system shall perform semantic internship matching in the background using FAISS and return the top- <i>k</i> results with an LLM-generated rationale.
FR-06	The system shall generate a personalised cover letter (Gemini) and a skill-gap analysis (Groq) for any selected internship on demand.
FR-07	The system shall provide a RAG-based product chatbot that answers platform-related queries using a FAISS-indexed knowledge base, with session history stored per user.
FR-08	The system shall provide an Interview Preparation Agent that classifies user intent (<i>role_recommendation</i> , <i>interview_prep</i> , <i>document_qa</i> , <i>general</i>) and responds accordingly.
FR-09	The interview agent shall accept uploaded PDF or DOCX documents and use the extracted text as additional context for question answering.
FR-10	The system shall persist and retrieve full chat history per session for both the product chatbot and the interview agent.
FR-11	The system shall allow users to log internship applications and update their status through a defined pipeline.

4.2 Non-Functional Requirements

ID	Requirement	Measurable Target
NFR-01	Response Time	Standard chat API responses in <5 s; cover letter in <10 s; matching background task completes within 30 s.
NFR-02	Security	Passwords hashed with bcrypt cost factor 12; no plaintext credentials stored; user-scoped endpoints validated via <code>_require_owner()</code> .
NFR-03	Scalability	Stateless FastAPI backend; FAISS index loaded at startup; each request is independently servable — horizontal scaling ready.
NFR-04	Usability	All primary user flows completable in ≤ 5 clicks; error messages are human-readable and surfaced in the Next.js UI.
NFR-05	Reliability	Groq LLM wrapped with automatic Gemini fallback; LLM failures do not crash the API — a graceful error response is always returned.
NFR-06	Maintainability	Backend modularised into <code>matching_engine</code> , <code>chat_engine</code> , and <code>interview_agent</code> ; model names centralised in <code>config.py</code> .

5 System Architecture / Workflow

5.1 High-Level Architecture

I kept the architecture deliberately simple: a three-tier system with a clear separation of concerns. The **Next.js 14** frontend only talks to the **FastAPI** backend via REST — it never touches the database or the LLMs directly. The backend owns everything AI-related: two FAISS vector stores, two LLM providers (Groq as primary, Gemini as automatic fallback), and a single **PostgreSQL** database for users, resumes, chat history, and applications. Keeping it this way meant I could debug, swap, or upgrade any individual piece without touching the others.

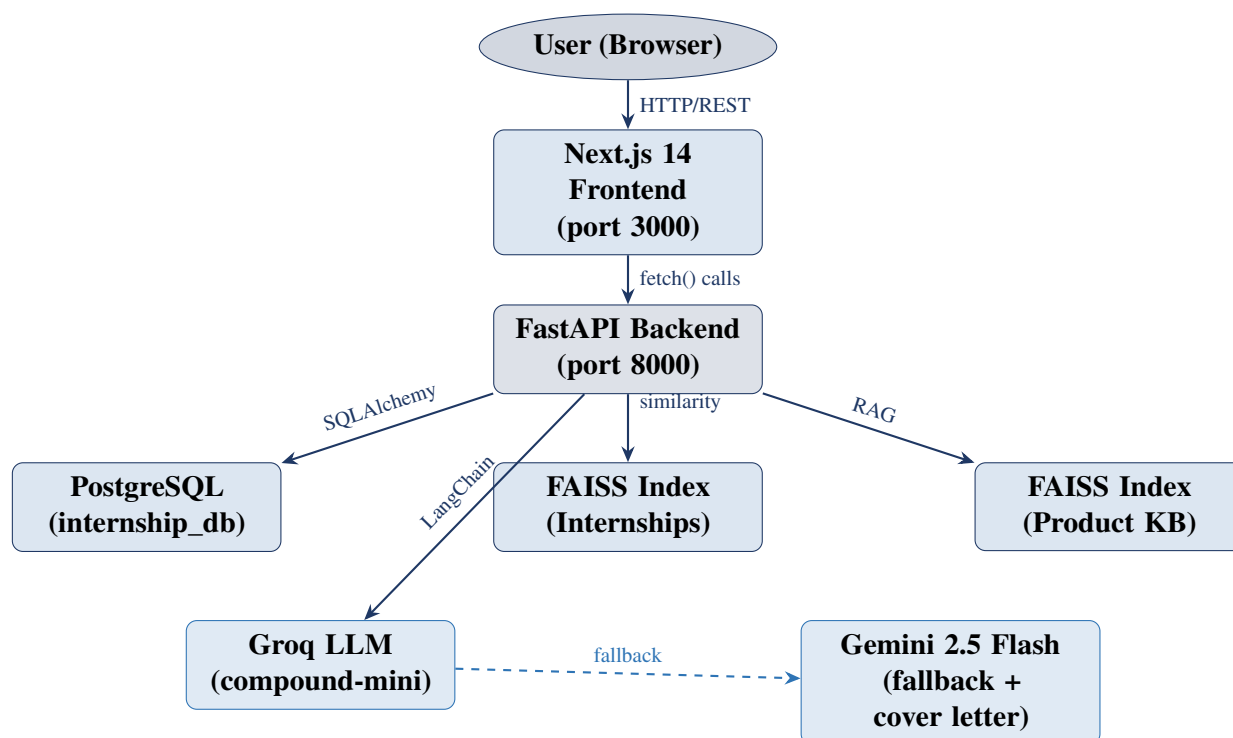


Figure 1: High-Level System Architecture

5.2 Resume Upload & Matching Flow

One of the first UX decisions I had to make: the FAISS search + LLM matching combo takes 5–15 seconds. Blocking the upload endpoint for that long would feel broken to any user. So I split it into two phases — the endpoint parses the resume synchronously (fast, 2–3 s) and returns immediately with a `202 Accepted`, then hands off the expensive FAISS + LLM work to a **FastAPI BackgroundTask**. The frontend polls `/api/analysis_status` and shows a progress bar until the result lands in `Resume.match_result`.

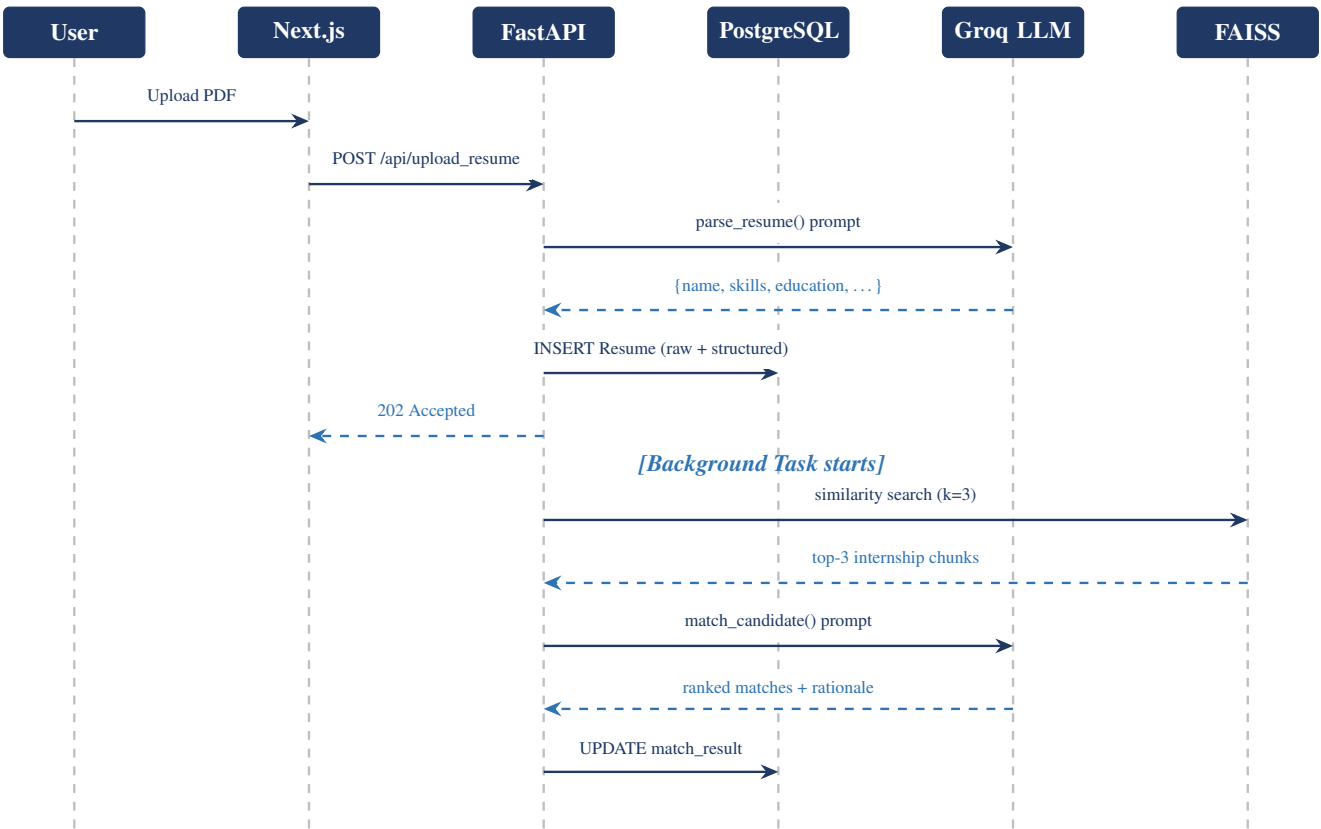


Figure 2: Resume Upload & Matching – Sequence Diagram

5.3 RAG / AI Processing Workflow

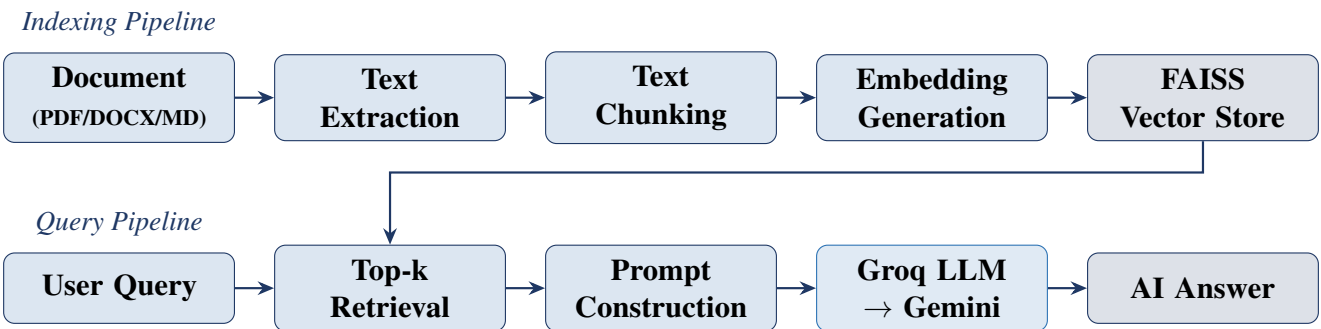


Figure 3: RAG / AI Workflow – Indexing and Query Pipelines

5.4 Interview Preparation Agent – Intent Flow

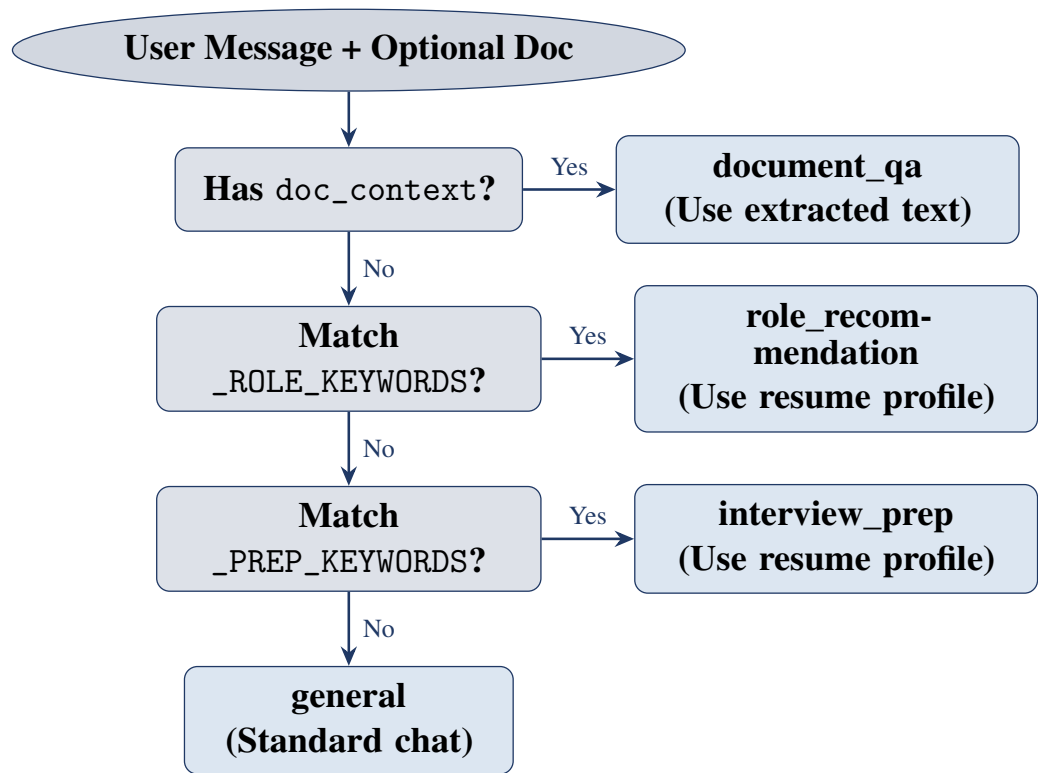


Figure 4: Interview Agent – Intent Classification Flow

5.5 Application Tracker Flow

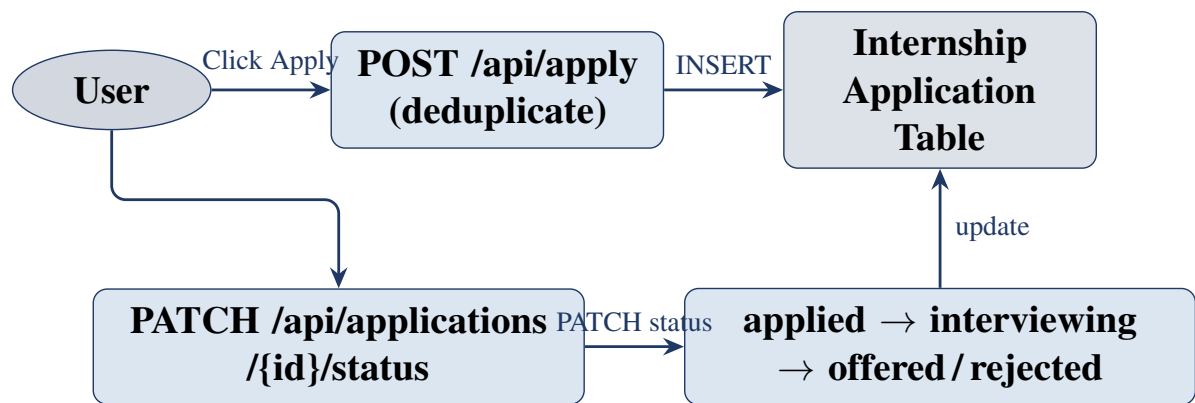


Figure 5: Application Tracker – Status Pipeline

6 Product Backlog

The backlog lists all features and tasks prioritised by business value. Items are ordered by dependency and priority.

ID	Feature / Task	Priority	Status
P001	Project Setup (repo, virtualenv, folder structure)	High	✓ Done
P002	PostgreSQL database setup + SQLAlchemy ORM models	High	✓ Done
P003	User Authentication (register + login + bcrypt)	High	✓ Done
P004	Resume upload endpoint + PyMuPDF PDF extraction	High	✓ Done
P005	LLM resume parsing → structured JSON	High	✓ Done
P006	FAISS internship vector store build from JSON dataset	High	✓ Done
P007	Internship matching engine (FAISS + LLM rationale)	High	✓ Done
P008	Cover letter generation (Gemini)	High	✓ Done
P009	Skill-gap analysis (Groq)	High	✓ Done
P010	Product chatbot RAG (product_knowledge.md → FAISS)	Medium	✓ Done
P011	Chat session management + history persistence	Medium	✓ Done
P012	Interview Prep Agent (multi-intent, resume-aware)	High	✓ Done
P013	Document upload for interview agent (PDF/DOCX)	High	✓ Done
P014	Interview session management + history	Medium	✓ Done
P015	Application tracker (POST apply + status PATCH)	Medium	✓ Done
P016	Next.js dashboard (resume upload, matches, chatbot)	High	✓ Done
P017	Interview Prep frontend page	High	✓ Done
P018	Full test suite (API + feature + integration)	High	✓ Done
P019	Production deployment configuration	Low	✓ Done

7 Sprint Planning

Nine weeks, nine sprints — I ran the project on a strict one-week cadence from 22 July to 22 September 2026. I genuinely found Agile useful here, not just as a process checkbox: breaking the work into weekly goals forced me to ship something tangible each week instead of endlessly designing in my head. The first three sprints were all infrastructure and plumbing (boring but essential), and I’m glad I front-loaded that pain — by Sprint 5 I could focus entirely on features without hitting database or auth issues mid-sprint.

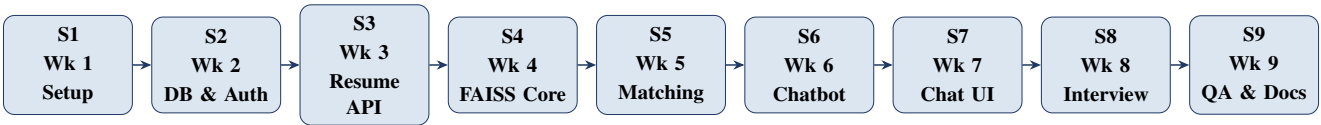


Figure 6: Sprint Timeline Overview – 9 Sprints (22 Jul – 22 Sep 2026)

Sprint	Dates	Goal	Key Tasks
Sprint 1	22 Jul – 28 Jul	Project Setup & Planning	Analyse requirements and assignment brief; design folder structure; initialise Git repository; configure Python virtual environment and Next.js 14 project; plan DB schema and API contract.
Sprint 2	29 Jul – 4 Aug	Database & Auth	Define SQLAlchemy ORM models (User, Resume, ChatMessage, InternshipApplication); configure PostgreSQL (internship_db); implement POST /api/register and POST /api/login with bcrypt password hashing.
Sprint 3	5 Aug – 11 Aug	Resume Upload API	Build POST /api/upload_resume multipart endpoint; integrate PyMuPDF (fitz) for PDF text extraction; implement Groq LLM resume parser producing structured JSON; add scanned-PDF heuristic guard.
Sprint 4	12 Aug – 18 Aug	FAISS Core & Data	Prepare internship_data.json dataset; build FAISS internship vector index using HuggingFace Sentence Transformers; implement InternshipMatcher.get_raw_retrieval(); scaffold matching_engine.py and llm_client.py with Groq → Gemini fallback via LangChain.with_fallbacks().
Sprint 5	19 Aug – 25 Aug	Matching & Insights	Complete match_candidate() LCEL chain (retriever → prompt → LLM); move FAISS matching to FastAPI BackgroundTasks; implement GET /api/analysis_status polling; add POST /api/generate_insights for cover letter (Gemini) and skill-gap analysis (Groq); build initial Next.js dashboard with resume upload and match display.
Sprint 6	26 Aug – 1 Sep	Product Chatbot (RAG)	Index product_knowledge.pdf into a FAISS product vector store via doc_indexer.py (PyMuPDFLoader, chunk size 500); implement ProductChatbot.chat() with RAG + conversation memory; add POST /api/chat/session, POST /api/chat, GET /api/chat/history and POST /api/chat/feedback endpoints; integrate chatbot into the dashboard sidebar.
Sprint 7	2 Sep – 8 Sep	Chat UI & UX Polish	Build full chatbot UI with session switcher, source-chunk display, and like/dislike feedback; add dark/light mode toggle; refine signup and profile pages; resolve backend chatbot bugs identified during internal testing.

Sprint	Dates	Goal	Key Tasks
Sprint 8	9 Sep – 15 Sep	Interview Prep Agent & Tracker	Implement InterviewPrepAgent (agent.py) with keyword-based intent detection (role recommendation, interview prep, document Q&A, general); build doc_parser.py for PDF/DOCX extraction; add all /api/interview/* endpoints; implement application tracker (POST /api/apply, PATCH /api/applications/{id}/status) with deduplication; build Interview Prep frontend page and landing page.
Sprint 9	16 Sep – 22 Sep	Testing, QA & Documentation	Execute full test suite (T001–T016) covering auth, resume, matching, chatbot, interview agent, and tracker flows; perform negative and integration testing; fix remaining UI bugs; write Agile project documentation.

8 Sprint Tasks & Status

8.1 Task Kanban Flow



Figure 7: Kanban Task Lifecycle

8.2 Task Tracker

Sprint	Task	Status
Sprint 1	Requirements analysis & project planning	✓ Done
Sprint 1	Git repo init, folder structure, virtual environment setup	✓ Done
Sprint 1	Next.js 14 project scaffold + Tailwind CSS	✓ Done
Sprint 2	SQLAlchemy ORM models (User, Resume, ChatMessage, InternshipApplication)	✓ Done
Sprint 2	PostgreSQL (internship_db) configuration + database.py	✓ Done
Sprint 2	FastAPI auth routes: POST /api/register & POST /api/login	✓ Done
Sprint 3	POST /api/upload_resume multipart endpoint	✓ Done
Sprint 3	PyMuPDF PDF text extraction + scanned-PDF guard	✓ Done
Sprint 3	Groq LLM resume parser → structured JSON storage	✓ Done
Sprint 4	Internship dataset preparation (internship_data.json)	✓ Done

Sprint	Task	Status
Sprint 4	FAISS internship vector index build (HuggingFace embeddings)	✓ Done
Sprint 4	llm_client.py: Groq primary → Gemini fallback via LangChain	✓ Done
Sprint 5	InternshipMatcher: FAISS retrieval + LLM LCEL rationale chain	✓ Done
Sprint 5	Async matching via FastAPI BackgroundTasks + status polling	✓ Done
Sprint 5	Cover letter generation (Gemini) + skill-gap analysis (Groq)	✓ Done
Sprint 5	Initial Next.js dashboard: resume upload + match display	✓ Done
Sprint 6	product_knowledge.md → FAISS product index (doc_indexer.py)	✓ Done
Sprint 6	ProductChatbot.chat() RAG pipeline + conversation memory	✓ Done
Sprint 6	Chatbot endpoints: session, chat, history, feedback	✓ Done
Sprint 7	Chatbot UI: session switcher, source-chunk display, feedback	✓ Done
Sprint 7	Dark/light mode toggle; signup & profile page refinement	✓ Done
Sprint 7	Backend chatbot bug fixes identified in internal testing	✓ Done
Sprint 8	InterviewPrepAgent: intent detection + prompts (prompts.py)	✓ Done
Sprint 8	doc_parser.py: PDF/DOCX text extraction for document Q&A	✓ Done
Sprint 8	All /api/interview/* endpoints + session isolation	✓ Done
Sprint 8	Application tracker: POST /api/apply + PATCH status + deduplication	✓ Done
Sprint 8	Interview Prep frontend page + landing page	✓ Done
Sprint 9	Full test suite T001–T016 (auth, resume, matching, chatbot, interview, tracker)	✓ Done
Sprint 9	Negative & integration testing; remaining bug fixes; UI polish	✓ Done
Sprint 9	Agile project documentation	✓ Done

9 Implementation / Key Features

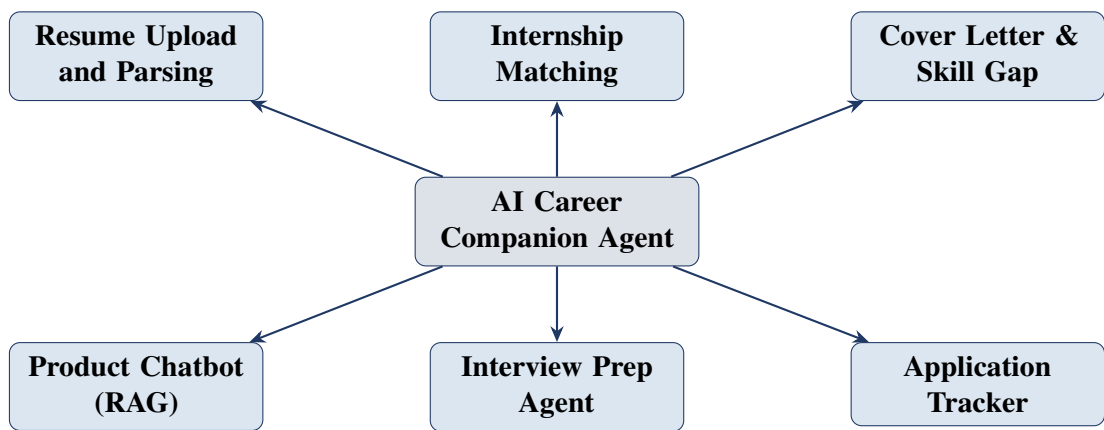


Figure 8: Feature Overview

9.1 Feature 1: User Authentication

Item	Detail
Description	Users register with email and password and log in to receive a persistent user_id. Passwords are never stored in plaintext.
Endpoints	POST /api/register POST /api/login
Implementation	FastAPI route → SQLAlchemy query on User table → passlib[bcrypt] for hashing (cost factor 12) → returns {message, user_id}.
Session Handling	user_id stored in browser localStorage; no JWT required; user-scoped endpoints guarded by _require_owner().
Output	Successful registration: {"message": "User registered successfully", "user_id": 3}. Successful login: {"message": "Login successful", "user_id": 3}. Duplicate email returns HTTP 400.

9.2 Feature 2: Resume Upload & LLM Parsing

Item	Detail
Description	User uploads a PDF resume. The system extracts raw text and uses a Groq LLM to parse it into a structured JSON profile.
Endpoint	POST /api/upload_resume?user_id=N (multipart/form-data)
Extraction	PyMuPDF (<code>fitz.open()</code>) iterates pages and concatenates <code>page.get_text()</code> . Handles multi-column layouts and embedded fonts.
LLM Parsing	Groq prompt instructs the model to return strict JSON with keys: <code>name</code> , <code>skills[]</code> , <code>education</code> , <code>experience</code> , <code>projects</code> . JSON is sanitised and validated before storage.
Storage	<code>Resume.raw_text</code> (full PDF text) and <code>Resume.structured_data</code> (JSON string) saved to PostgreSQL. Only one resume can be <code>is_active = True</code> per user.
Background Task	FAISS matching triggered via FastAPI <code>BackgroundTasks</code> immediately after the synchronous parse step. Progress polled at GET /api/analysis_status/{user_id}.
Output	<pre>{"message": "Resume parsed - matching is running in background.", "extracted_data": {"name": "Rishi Yadav", "skills": ["Python", "FastAPI", ...], "education": "B.Tech CS", ...}}.</pre>

9.3 Feature 3: Internship Matching (FAISS + LLM)

Item	Detail
Description	Semantically matches the candidate's profile against internship listings and returns the top- <i>k</i> results with a natural-language ranking rationale.
Endpoint	GET /api/matches/{user_id}
FAISS Retrieval	<code>InternshipMatcher.get_raw_retrieval(candidate_data, k=3)</code> queries the HuggingFace-embedded internship FAISS index.
LLM Rationale	<code>match_candidate()</code> builds a LangChain LCEL chain: retriever → prompt template → Groq LLM → output parser. Returns a formatted string with ranked matches and reasoning.
Caching	Result stored in <code>Resume.match_result</code> ; subsequent GET /api/matches reads the cache unless a retry is requested.
Output	Returns <code>raw_matches</code> (list of company, title, L2 distance, relevance score 0–100%) and <code>llm_rationale</code> (natural-language paragraph explaining the best-fit internship and why).

9.4 Feature 4: Cover Letter & Skill-Gap Generation

Item	Detail
Description	For a selected internship, generates a tailored cover letter and identifies skills the candidate should develop to strengthen their application.
Endpoint	POST /api/generate_insights Body: {user_id, company, title}
Cover Letter	generate_cover_letter() queries Gemini 2.5 Flash with the candidate's structured profile + the target company and role. Gemini produces more fluent, human-quality prose than compact fast models.
Skill Gap	generate_skill_gap() uses Groq to compare the candidate's skills against the internship requirements and returns a bulleted improvement plan.
Output	{"cover_letter": "Dear Hiring Manager, ...", "skill_gap": "• Strengthen proficiency in Docker\n• Learn cloud deployment (AWS/GCP)\n..."}.

9.5 Feature 5: Product Chatbot (RAG)

Item	Detail
Description	A conversational chatbot that answers questions about the platform using retrieval-augmented generation over the product_knowledge.pdf knowledge base. Full session history is maintained in PostgreSQL.
Endpoints	POST /api/chat/session POST /api/chat GET /api/chat/history/{user_id}/{session_id}
RAG Pipeline	doc_indexer.py loads product_knowledge.pdf with PyMuPDFLoader, splits it into chunks (size 500, overlap 50) using RecursiveCharacterTextSplitter, and builds the FAISS product index. ProductChatbot.chat() retrieves the top-3 chunks most relevant to the user query and passes them as context to Groq LLM.
Memory	Last 10 messages (HISTORY_WINDOW=10) loaded from the ChatMessage table and prepended to each prompt. Source chunks stored per assistant message for provenance.
Output	{"response": "The platform supports PDF resume upload ...", "session_id": "uuid", "source_chunks": [{"content": "...", "source": "product_knowledge.md"}]}.

9.6 Feature 6: Interview Preparation Agent

Item	Detail
Description	A multi-intent AI agent that provides role recommendations, generates technical and HR interview questions with model answers, and answers questions about uploaded reference documents.
Endpoints	POST /api/interview/session POST /api/interview/chat POST /api/interview/upload_doc GET /api/interview/history/{user_id}/{session_id}
Intent Detection	_detect_intent() in agent.py classifies each message using keyword lists: _ROLE_KEYWORDS and _PREP_KEYWORDS. If doc_context is non-empty the intent is forced to document_qa; otherwise keywords matched case-insensitively against the user's query string determine role_recommendation, interview_prep, or general. No secondary LLM call is made for classification.
Resume Context	The active resume's structured_data is always injected into the context so role recommendations and Q&A are personalised to the candidate.
Document Q&A	doc_parser.py extracts text from uploaded PDFs/DOCX files and passes it as doc_context into the agent prompt for document-grounded answers.
Session Isolation	All interview sessions are prefixed "interview-<uuid>" in the ChatMessage table to distinguish them from product chatbot sessions.
Output	{"response": "Based on your skills in Python and FastAPI, I recommend applying for Backend Engineering roles...", "session_id": "interview-uuid"}.

9.7 Feature 7: Application Tracker

Item	Detail
Description	Users can record internship applications from the match results and track them through a four-stage status pipeline.
Endpoints	POST /api/apply GET /api/applications/{user_id} PATCH /api/applications/{id}/status
Deduplication	The POST /api/apply endpoint checks for an existing record with the same (user_id, company, title) before inserting; duplicate clicks are silently ignored.
Status Pipeline	applied → interviewing → offered or rejected. Status is validated server-side against the allowed enum.
Output	POST /api/apply: {"message": "Application recorded", "application_id": 5, "already_applied": false}. GET /api/applications: list of all applications with status, company, title, and timestamps.

10 Testing

10.1 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
T001	Register with valid email and password	Account created, user_id returned	User registered	✓ Pass
T002	Register with duplicate email	HTTP 400 error returned	Error message shown	✓ Pass
T003	Login with valid credentials	user_id returned	User logged in	✓ Pass
T004	Login with wrong password	HTTP 400 returned	Invalid credentials error	✓ Pass
T005	Upload valid PDF resume	Resume parsed; background matching started	Resume saved; analysis status updated	✓ Pass
T006	Upload a non-PDF file	HTTP 400 validation error	Error message returned	✓ Pass
T007	Fetch internship matches after upload	Top-3 matches with rationale returned	Matches displayed in UI	✓ Pass
T008	Generate cover letter for a selected internship	Personalised cover letter text generated	Cover letter displayed	✓ Pass
T009	Generate skill-gap analysis	Bulleted improvement plan returned	Skill gap list shown	✓ Pass
T010	Ask a question to the product chatbot	Relevant RAG-grounded answer with source chunks	Answer returned correctly	✓ Pass
T011	Start an interview prep session	Session ID with interview- prefix created	Session initialised	✓ Pass
T012	Ask role recommendation question	Roles recommended based on active resume	Relevant roles listed	✓ Pass
T013	Upload DOCX to interview agent and ask question	Answer grounded in uploaded document content	Correct contextual answer	✓ Pass
T014	Record an internship application	Application saved; visible in tracker	Application logged	✓ Pass
T015	Update application status to <i>interviewing</i>	Status updated in DB; reflected in UI	Status changed correctly	✓ Pass
T016	Duplicate application attempt	No duplicate row created; silent 200 response	Deduplication worked	✓ Pass

10.2 Testing Types Performed

Testing Type	Description
Functional Testing	Each feature verified against its corresponding functional requirement (FR-01 through FR-11) using manual test execution.
API Testing	All FastAPI endpoints tested with Postman collections; request/response payloads validated against expected schemas.
UI Testing	Next.js dashboard and interview page verified for correct rendering, form validation, error display, and responsive layout.
Integration Testing	Full end-to-end flows tested: upload resume → parse → match → generate cover letter; and: start session → upload doc → ask Q&A.
AI / RAG Testing	FAISS retrieval accuracy evaluated on 10 test resumes; LLM response quality assessed for relevance, hallucination, and formatting.
Negative Testing	Invalid inputs (wrong file type, wrong credentials, duplicate records) tested to verify correct HTTP error responses are returned.

Testing Note

LLM response quality is inherently non-deterministic. AI/RAG test cases were evaluated using qualitative rubrics (relevance, groundedness, format compliance) rather than exact string matching.

11 Challenges & Solutions

Honest Mistakes

These aren't textbook challenges I picked from a list — they're the actual moments where things broke and I had to figure out why. I've tried to explain not just what I did to fix each one, but why the problem happened in the first place.

#	Challenge	Solution
1	Malformed JSON from LLM during resume parsing. Groq occasionally returned partial JSON or wrapped it in markdown code fences.	Added a JSON extraction and sanitisation step before <code>json.loads()</code> . If parsing still fails after sanitisation, a retry is issued with a stricter prompt that instructs the model to output <i>only</i> the JSON object and nothing else.
2	FAISS returning irrelevant results for sparse resumes. Resumes with fewer than five skills produced poor semantic embeddings, causing the top- <i>k</i> results to be off-target.	Enriched the candidate query string by combining all structured fields (skills, education, experience) into a single dense paragraph before embedding, rather than embedding the raw resume text.
3	Resume upload blocking on long LLM parse. The synchronous Groq parse call added 3–6 seconds to the upload response time.	Kept the parse step synchronous (required for storing <code>structured_data</code>) but moved the FAISS matching step to a FastAPI <code>BackgroundTask</code> . The UI polls <code>/api/analysis_status</code> and shows a progress indicator.
4	Interview and product chatbot sessions mixing in DB. Both used the <code>ChatMessage</code> table; queries without a prefix filter returned interleaved session lists.	Prefixed all interview session IDs with <code>"interview-<uuid>"</code> . Session list queries filter on <code>session_id LIKE 'interview-%'</code> or exclude them, depending on the endpoint.
5	Gemini fallback not triggering reliably. LangChain's <code>.with_fallbacks()</code> was being called on the raw Groq object instead of the LangChain wrapper, so the fallback chain was never invoked.	Ensured <code>get_llm()</code> returns a <code>ChatGroq</code> instance (a <code>LangChain BaseChatModel</code>) and the fallback is a <code>ChatGoogleGenerativeAI</code> instance, both conforming to the same interface. Tested by temporarily revoking the Groq API key.
6	PyMuPDF producing garbled text for scanned / image PDFs. Some resumes were scanned documents with no embedded text layer.	Added a character-count heuristic: if extracted text is fewer than 100 characters, a warning is returned asking the user to upload a text-based PDF. OCR integration is listed as a future enhancement.

12 Final Project Status

12.1 Completed Features

	Feature
✓ Done	User Registration & Login (bcrypt hashing, persistent user_id session)
✓ Done	Resume Upload, PyMuPDF Text Extraction & Groq LLM Parsing
✓ Done	FAISS Internship Vector Store Build & Index
✓ Done	Internship Matching Engine (FAISS + LLM Rationale, top-k)
✓ Done	Cover Letter Generation (Gemini 2.5 Flash)
✓ Done	Skill-Gap Analysis (Groq)
✓ Done	Product Chatbot — RAG over product_knowledge.md
✓ Done	Chat Session Management & History Persistence
✓ Done	Interview Prep Agent (Role Recommendation + Mock Q&A + Document Q&A)
✓ Done	Document Upload for Interview Agent (PDF / DOCX)
✓ Done	Application Tracker with Status Pipeline (Applied → Interviewing → Offered/Rejected)
✓ Done	Next.js 14 Dashboard (Resume Upload, Matches, Product Chatbot)
✓ Done	Interview Preparation Frontend Page
✓ Done	Full Test Suite (API + Integration + AI/RAG, T001–T016)
✓ Done	UI Polish, Dark/Light Mode & Comprehensive Error Handling
✓ Done	Production Deployment & Environment Configuration

12.2 Final Outcome

Honestly, I'm happy with how this turned out. Going in, I wasn't sure whether nine weeks was enough to build something that actually works end-to-end with real LLMs and a real database — it is, but only because I kept scope tight and made deliberate trade-offs.

A user can now:

- Drop in a PDF resume and get back a structured profile plus three personalised internship matches with a full AI-written rationale — all within about 30 seconds.
- Click on any match and instantly get a tailored cover letter and a specific, actionable skill-gap breakdown.
- Have a proper multi-turn conversation with a product chatbot that actually knows what the platform does (RAG keeps it grounded — it doesn't hallucinate features).
- Run a full interview prep session where the AI generates questions based on their actual resume, not some generic template; upload a job description PDF and ask questions about it on the spot.
- Track every application they send out and move it through a pipeline — applied to interviewing to offered or rejected.

All 19 backlog items shipped across 9 sprints. The test suite (T001–T016) passed clean. The three biggest lessons I’m taking away: moving the FAISS matching into a BackgroundTask was the single best UX decision I made; building a JSON sanitisation layer early saved me hours of debugging Groq’s occasionally chatty output; and embedding all structured resume fields together (not just the raw text) made FAISS retrieval noticeably more relevant. If I were starting over, I’d add that embedding strategy on day one instead of discovering it in Sprint 5.

Repository

The complete source code is publicly available on GitHub:

<https://github.com/RishiY7/AI-Career-Companion-Agent>